

SPRINT 2: SQL

NIVEL 1	1
EJERCICIO 1	1
EJERCICIO 2.....	3
EJERCICIO 3.....	4
EJERCICIO 4.....	6
EJERCICIO 5.....	9
EJERCICIO 6.....	10
EJERCICIO 7.....	11
EJERCICIO 8.....	12
EJERCICIO 9.....	14
EJERCICIO 10.....	14
NIVEL 2	14
EJERCICIO 1	14
EJERCICIO 2.....	14
EJERCICIO 3.....	14
EJERCICIO 4.....	14
EJERCICIO 5.....	14

NIVEL 1

EJERCICIO 1

```

1 • USE Transaction;
2 -- Ejercicio 1
3 -- Creamos la base de datos
4
5 • CREATE DATABASE IF NOT EXISTS transactions;
6 Use transactions;
7 -- Creamos la tabla company
8 • CREATE TABLE IF NOT EXISTS company (
9     id VARCHAR(15) PRIMARY KEY,
10    company_name VARCHAR(255),
11    phone VARCHAR(15),
12    email VARCHAR(100),
13    country VARCHAR(100),
14    website VARCHAR(255)
15 );
16
17 -- Creamos la tabla transaction
18 • CREATE TABLE IF NOT EXISTS transaction (
19     id VARCHAR(255) PRIMARY KEY,
20     credit_card_id VARCHAR(15) REFERENCES credit_card(id),
21     company_id VARCHAR(20),
22
23 );
24
25 FOREIGN KEY (company_id) REFERENCES company(id)
26 );
27
28 -- Insertamos datos de company desde N1-Ex.1_dades_introduir.
29
30
31 • SELECT*FROM transaction;
32
33 -- Ejercicio 2

```

Output

#	Time	Action	Message	Duration / Fetch
5135	11:03:17	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.015 sec
5136	11:03:19	Use transactions	0 row(s) affected	0.015 sec
5137	11:03:23	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, compa...	0 row(s) affected	0.016 sec
5138	11:03:29	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, cred...	0 row(s) affected	0.032 sec

Result Grid

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-494B-A5DD-A5BAE3BB190D	CcS-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D6B-ED2E-4F2F-8186-CEE074D8750D	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-8070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-8C99-E9C92EC3CA4D	CcU-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2928	51.9362	5.34265	2023-12-31 00:06:36	383.63	0
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	3902	45.492	-73.5706	2017-07-18 07:52:02	197.80	0
000A1DEC-CD86-4AB2-A619-71DA8BD4A262	CcS-6467	b-2558	1886	55.7425	-3.30009	2018-09-08 05:29:58	339.94	0
000A1E64-1414-40B0-9D92-5678A4D958E2	CcS-5966	b-2550	1385	52.0821	5.28424	2022-09-17 04:02:19	369.71	0
000A5879-3472-41D9-AF60-42D35038543C	CcU-4569	b-2590	42	39.949	-75.1719	2020-02-07 23:03:45	162.43	0
000AE0D4-F06E-4146-804A-5A2FC73110D7	CcS-8134	b-2426	3553	46.8055	-71.2149	2017-07-10 07:11:41	188.94	0
000BCBF8-A44B-4EBE-A148-2E7A04E1B385	CcS-6020	b-2598	1439	33.4472	-112.072	2019-05-28 04:23:02	96.22	0

transaction 1 x

Output

#	Time	Action	Message	Duration / Fetch
105239	11:09:03	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, am...	1 row(s) affected	0.016 sec
105240	11:09:03	USE transactions	0 row(s) affected	0.000 sec
105241	11:09:03	INSERT INTO company (id, company_name, phone, email, country, website) VALUES (b-2...	Error Code: 1062. Duplicate entry 'b-2222' for key 'company.PRIMARY'	0.000 sec
105242	11:14:30	SELECT*FROM transaction	100000 row(s) returned	0.016 sec / 0.234 sec

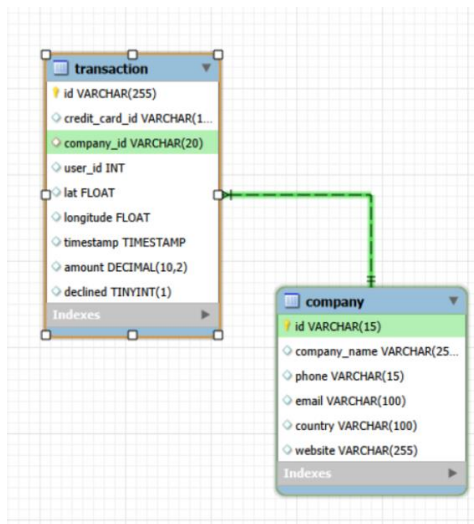
Iniciamos creando la base de datos transactions, y las dos tablas que encontraremos en la base de datos: Transaction y company. Insertaremos los datos de los archivos: N1-Ex.1__estructura dades y N1-Ex.1__dades_introduir. Ejecutaremos para comprobar su creación.

```
-- Creamos la tabla transaction
CREATE TABLE IF NOT EXISTS transaction (
  id VARCHAR(255) PRIMARY KEY,
  credit_card_id VARCHAR(15) REFERENCES credit_card(id),
  company_id VARCHAR(20),
  user_id INT REFERENCES user(id),
  lat FLOAT,
  longitude FLOAT,
  timestamp TIMESTAMP,
  amount DECIMAL(10, 2),
  declined BOOLEAN,
  FOREIGN KEY (company_id) REFERENCES company(id)
);
```

Durante la creación de las tablas usamos conceptos como Varchar (n), INT, FLOAT, TIMESTAMP, DECIMAL, BOOLEAN. Estos son tipos de datos ¿Qué nos indica cada tipo de dato?

- VARCHAR(n): Contiene cadenas de caracteres de longitud variable hasta 'n' caracteres. Útil para emails.
- CHAR(n): Contiene cadenas de longitud fija. Si el dato es más corto, se rellena con espacios. Se usa para datos de longitud predecible y constante.
- INT: Contiene números enteros.
- FLOAT: Contiene números decimales de punto flotante (aproximados). Nota: No recomendado para dinero.
- TIMESTAMP: Contiene fecha y hora combinadas.
- DECIMAL(x,y): Contiene números decimales con precisión exacta. Lo usamos para datos monetarios (amount) para evitar errores de redondeo.
- BOOLEAN: Contiene valores lógicos (verdadero/falso), que se guardan como 1 o 0.

Utilizamos REFERENCES para definir claves foráneas en la tabla de hechos, vinculándola con las tablas de dimensiones. Esto configura un modelo en estrella, donde transaction es la tabla central rodeada de sus dimensiones como company.



EJERCICIO 2

(Interpreto 'Venta' como aquellas transacciones donde amount es mayor a 0 y declined es igual a 0.)

Listado de países que generan venta.

```
32 • SELECT*FROM transaction;
33
34 -- Exercici 2
35
36 -- 2.1 Llistat dels països que estan generant vendes.
37 • SELECT DISTINCT country
38 FROM company
39 INNER JOIN transaction ON company.id=transaction.company_id
40 WHERE declined=0 ;
41
```

Result Grid

country
Netherlands
Sweden
Ireland
United States
Belgium
Canada

Result 5 x Read Only

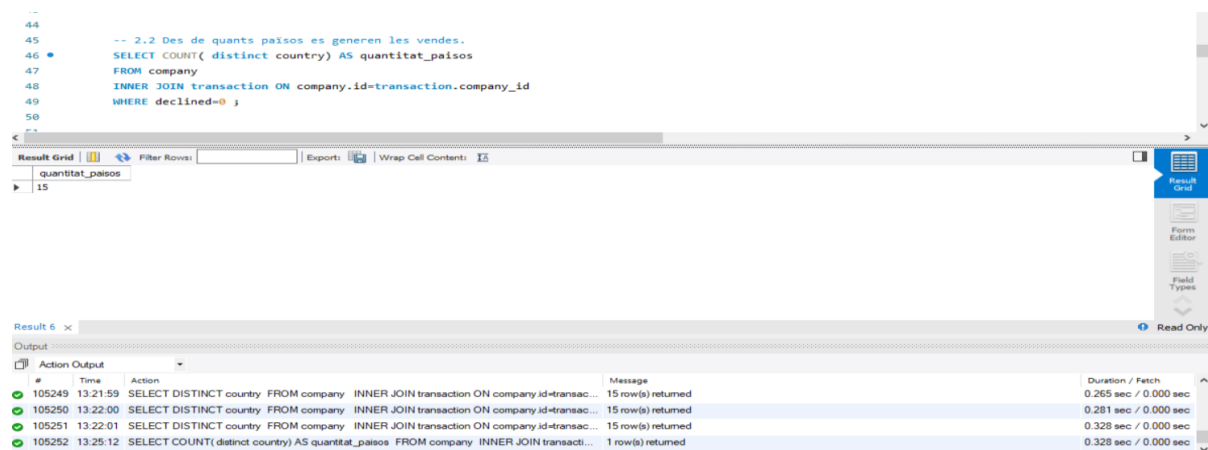
Output

#	Time	Action	Message	Duration / Fetch
105248	13:21:51	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.500 sec / 0.000 sec
105249	13:21:59	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.265 sec / 0.000 sec
105250	13:22:00	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.281 sec / 0.000 sec
105251	13:22:01	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.328 sec / 0.000 sec

Este código hace una lista única de los países que tienen al menos una venta exitosa. Para ello uno la tabla de empresas con la de transacciones. Filtro solo los casos donde la transacción no fue rechazada (declined=0). Finalmente, uso DISTINCT para que no se repita ningún país en la lista final.

Desde cuantos países se generan venta

```
--
44
45 -- 2.2 Des de quants països es generen les vendes.
46 • SELECT COUNT( distinct country) AS quantitat_països
47 FROM company
48 INNER JOIN transaction ON company.id=transaction.company_id
49 WHERE declined=0 ;
50
```

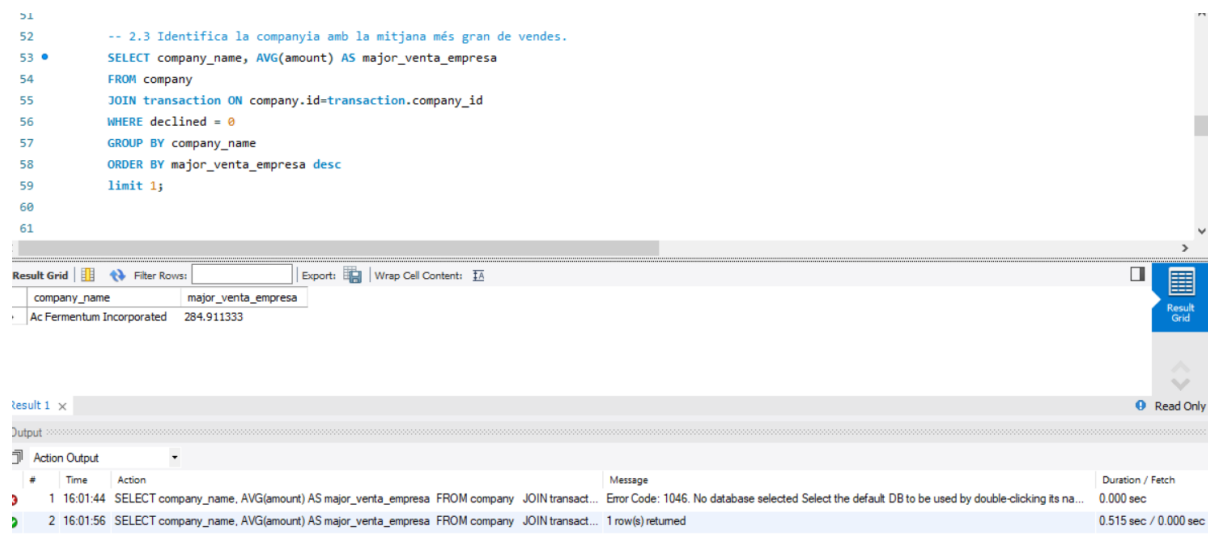


#	Time	Action	Message	Duration / Fetch
105249	13:21:59	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.265 sec / 0.000 sec
105250	13:22:00	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.281 sec / 0.000 sec
105251	13:22:01	SELECT DISTINCT country FROM company INNER JOIN transaction ON company.id=transac...	15 row(s) returned	0.328 sec / 0.000 sec
105252	13:25:12	SELECT COUNT(distinct country) AS quantitat_països FROM company INNER JOIN transacti...	1 row(s) returned	0.328 sec / 0.000 sec

Para este caso uso COUNT para indicar el número total de países activos. Luego uno la tabla de empresas con la de transacciones. Filtro solo los casos donde la transacción no fue rechazada (declined=0).

Identifica el país con la mediana más grande de venta

```
>1
52 -- 2.3 Identifica la companyia amb la mitjana més gran de vendes.
53 • SELECT company_name, AVG(amount) AS major_venta_empresa
54 FROM company
55 JOIN transaction ON company.id=transaction.company_id
56 WHERE declined = 0
57 GROUP BY company_name
58 ORDER BY major_venta_empresa desc
59 limit 1;
60
61
```



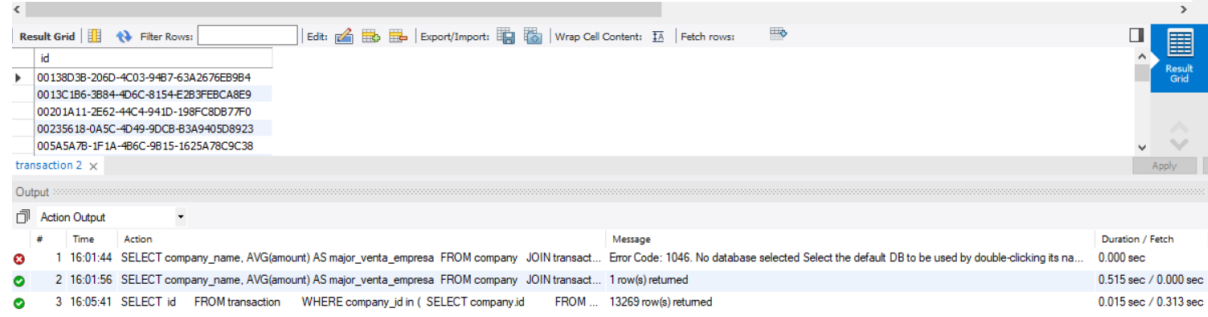
#	Time	Action	Message	Duration / Fetch
1	16:01:44	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its na...	0.000 sec
2	16:01:56	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	1 row(s) returned	0.515 sec / 0.000 sec

Para encontrar cuál es la empresa que más dinero mueve en promedio por transacción. Calculo el promedio (AVG) del amount de cada empresa. Solo considera las ventas no rechazadas. Finalmente, Ordena las empresas de mayor a menor promedio y con LIMIT 1 nos devuelve únicamente a la ganadora.

EJERCICIO 3

Muestra todas las transacciones realizadas por empresas de Alemania.

```
63 -- Exercici 3
64
65 -- 3.1 Mostra totes les transaccions realitzades per empreses d'Alemanya.
66
67 • SELECT id
68   FROM transaction
69  WHERE company_id in (
70     SELECT company_id
71     FROM company
72    WHERE country = 'Germany' and declined = 0);
73
```

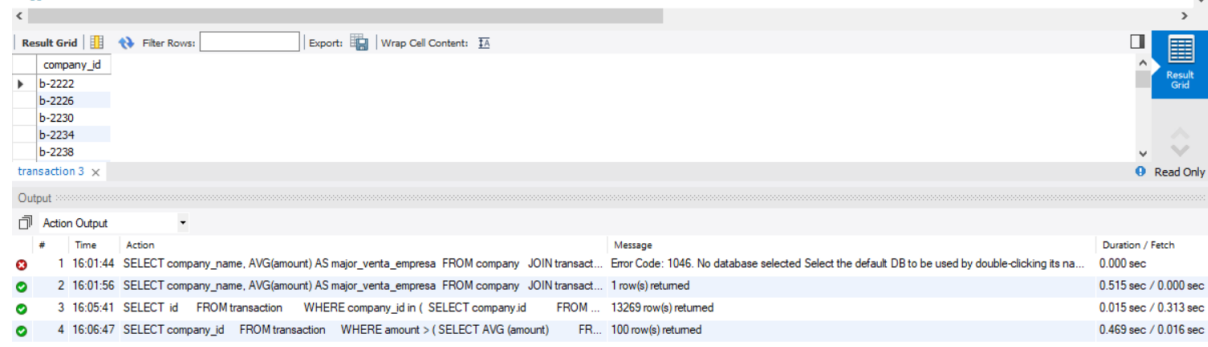


#	Time	Action	Message	Duration / Fetch
1	16:01:44	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its na...	0.000 sec
2	16:01:56	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	1 row(s) returned	0.515 sec / 0.000 sec
3	16:05:41	SELECT id FROM transaction WHERE company_id in (SELECT company_id FROM ...	13269 row(s) returned	0.015 sec / 0.313 sec

utilizo una subconsulta para encontrar primero las empresas alemanas. busco en la tabla company todas las que tienen country = 'Germany'. Luego, en la consulta principal, filtro la tabla transaction para quedarme solo con aquellas cuyo company_id coincida con los resultados de mi búsqueda anterior.

Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```
75 -- 3.2 Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
76
77 • SELECT company_id
78   FROM transaction
79  WHERE amount > (
80     SELECT AVG (amount)
81     FROM transaction
82     WHERE declined = 0
83   )
84  GROUP BY company_id;
85
```



#	Time	Action	Message	Duration / Fetch
1	16:01:44	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its na...	0.000 sec
2	16:01:56	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	1 row(s) returned	0.515 sec / 0.000 sec
3	16:05:41	SELECT id FROM transaction WHERE company_id in (SELECT company_id FROM ...	13269 row(s) returned	0.015 sec / 0.313 sec
4	16:06:47	SELECT company_id FROM transaction WHERE amount > (SELECT AVG (amount) FR...	100 row(s) returned	0.469 sec / 0.016 sec

Empiezo por la subquery, calculo la media (AVG) de todos los montos donde declined = 0. Luego, recorro la tabla transaction y selecciono aquellas filas donde el amount sea mayor que ese promedio calculado. Finalmente, agrupo (GROUP BY) por company_id para obtener una lista única de las empresas que tienen al menos una transacción por encima de la media.

Eliminarán del sistema las empresas que no tienen transacciones registradas; entrega la lista de dichas empresas.

The screenshot shows a SQL IDE interface. At the top, a SQL query is entered in a text area:

```
-- 3.3 Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.  
  
SELECT *  
FROM transaction  
WHERE company_id not in(  
  SELECT id  
  FROM company);
```

Below the query editor, a 'Result Grid' is displayed, showing a table with 9 columns: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, and declined. All cells in the first row are NULL.

At the bottom, the 'Action Output' pane shows the execution log for 'transaction 4':

#	Time	Action	Message	Duration / Fetch
1	16:01:44	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its na...	0.000 sec
2	16:01:56	SELECT company_name, AVG(amount) AS major_venta_empresa FROM company JOIN transact...	1 row(s) returned	0.515 sec / 0.000 sec
3	16:05:41	SELECT id FROM transaction WHERE company_id in (SELECT company.id FROM ...	13269 row(s) returned	0.015 sec / 0.313 sec
4	16:06:47	SELECT company_id FROM transaction WHERE amount > (SELECT AVG (amount) FR...	100 row(s) returned	0.469 sec / 0.016 sec
5	16:07:37	SELECT * FROM transaction WHERE company_id not in(SELECT id FROM company)	0 row(s) returned	0.141 sec / 0.000 sec

busco en la tabla transaction cualquier registro cuyo company_id no exista en la tabla company. Al obtener un resultado vacío (NULL), confirmo que la integridad referencial se ha respetado, y que por tanto cada transacción tiene una empresa válida asociada.

EJERCICIO 4


```

102 CREATE TABLE IF NOT EXISTS credit_card(
103   id VARCHAR (15) PRIMARY KEY,
104   iban VARCHAR (34),
105   pan VARCHAR (20),
106   pin VARCHAR (4),
107   cvv VARCHAR (4),
108   expiring_date CHAR (8),
109   expiring_date_ok DATE NULL
110 );
111
112 -- Cargar datos desde archivo NI-Ex-4__datos_introducir_credit.sql
113 -- Vinculamos la nueva tabla dimensión a la tabla de hechos.
114 ALTER TABLE transaction
115 ADD constraint FK2 foreign key (credit_card_id) REFERENCES credit_card(id);
116
117 -- Después de LA CREACIÓN. Convertir expiring_date a DATE
118 UPDATE credit_card
119 SET expiring_date_ok = STR_TO_DATE(expiring_date, "%m/%d/%y");
120
121 -- Eliminamos la columna original con mal formato.
122 ALTER TABLE credit_card
123 DROP COLUMN expiring_date,
124 CHANGE COLUMN expiring_date_ok expiring_date DATE ;

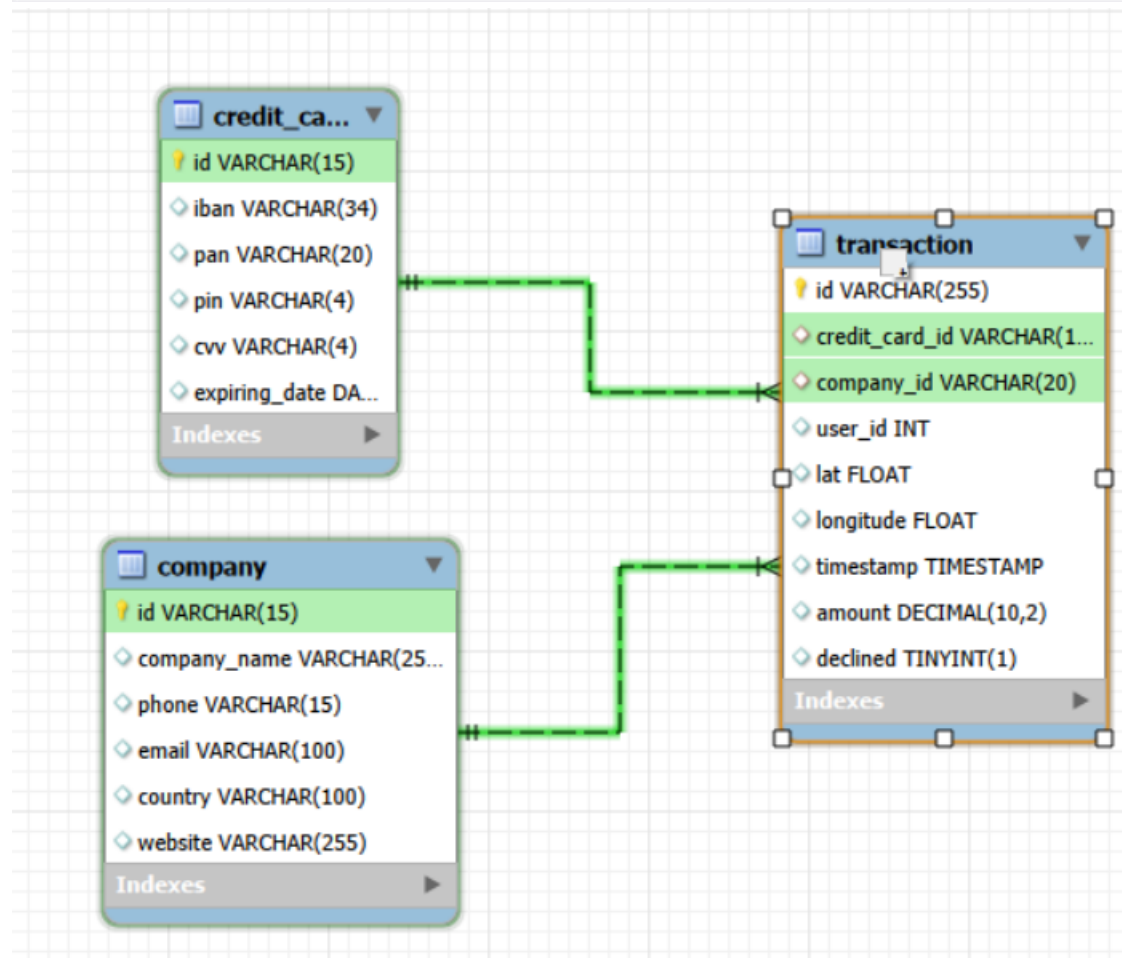
```

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	2025-09-27
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	2028-12-28
CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	2026-11-26
CcS-4860	XX555990368835304645299	2481155515498459	6876	661	2027-07-27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	2026-04-25
CcS-4862	XX4774721462463645409758	6715617009807829	4042	174	2026-11-27
CcS-4863	XX1476829664245046207111	3140879819451394	5969	449	2029-12-27
CcS-4864	XX8380298893385731196159	5793672133649114	8481	139	2026-02-28

credit_card 5

Output

#	Time	Action	Message	Duration / Fetch
5000	16:17:37	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX29639...	1 row(s) affected	0.000 sec
5001	16:17:37	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX78125...	1 row(s) affected	0.000 sec
5002	16:17:37	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX91567...	1 row(s) affected	0.000 sec
5003	16:17:37	SELECT * FROM Credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
5004	16:17:50	ALTER TABLE transaction ADD constraint FK2 foreign key (credit_card_id) REFERENCES cre...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.125 sec
5005	16:17:53	UPDATE credit_card SET expiring_date_ok = STR_TO_DATE(expiring_date, "%m/%d/%y")	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.234 sec
5006	16:18:04	ALTER TABLE credit_card DROP COLUMN expiring_date, CHANGE COLUMN expiring_dat...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
5007	16:19:21	SELECT * FROM credit_card	5000 row(s) returned	0.000 sec / 0.000 sec



Creo la tabla `credit_card` definiendo `id` como clave PK (`VARCHAR(15)`).

Incluyo los campos sensibles y necesarios para la validación: `iban`, `pan`, `pin`, `cvv`.

Para la fecha de caducidad, decido guardarla inicialmente como `CHAR(8)` porque los datos de entrada vienen en un formato que SQL no acepta.

Añado una columna temporal `expiring_date_ok` de tipo `DATE NULL`, preparada para recibir el dato limpio antes de eliminar el original.

Vínculo con la tabla de hechos (`transaction`) para asegurar la integridad del modelo de datos conectando mis nuevas tablas de dimensión con la tabla de hechos.

Utilizo `ALTER TABLE` sobre la tabla `transaction` para añadir una restricción de clave foránea (FK2). De esta forma, obligo a que cada transacción (`credit_card_id`) tenga que apuntar obligatoriamente a una tarjeta que exista realmente en mi nueva tabla `credit_card`.

Ejecuto un proceso de limpieza para convertir mis datos crudos en un formato de fecha estandarizado y útil.

Actualizo la tabla `credit_card` convirtiendo el texto (`expiring_date`) en una fecha real (`DATE`) usando `STR_TO_DATE` para manejar el formato "mes/día/año".

Finalmente, elimino la columna de texto original que ya no sirve y renombro la columna limpia (`expiring_date_ok`) por el nombre original (`expiring_date`).

Mi modelo ahora es un modelo en estrella completo:

Tabla (Hechos): `transaction` (registra la acción).

Tablas de Dimensión:

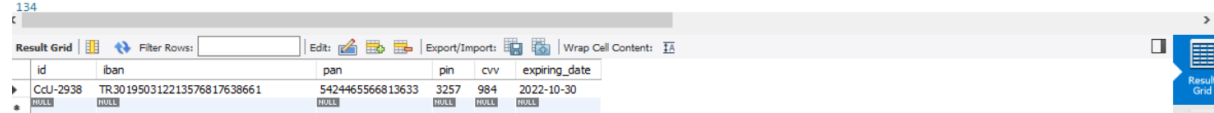
`company` (información del lugar/comercio).

`credit_card` (información del medio de pago).

La tabla `transaction` se conecta a las tres dimensiones mediante claves foráneas (`company_id`, `credit_card_id`), permitiendo analizar las ventas desde cualquier ángulo.

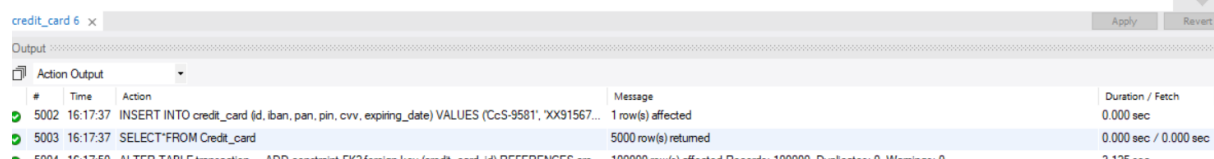
EJERCICIO 5

```
126 -- Ejercici 5
127 -- El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938.
128 -- La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.
129
130 -- Registro antes del cambio
131 • SELECT *
132 FROM credit_card
133 WHERE id='CcU-2938';
134
```



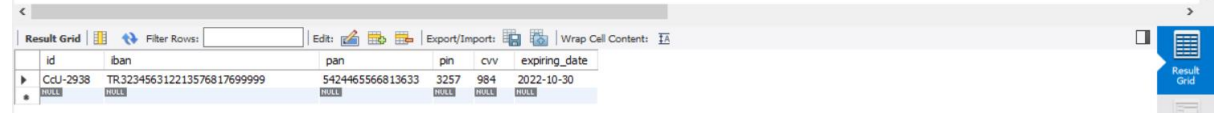
id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30

```
135
136 -- Ejecución del cambio.
137 • UPDATE credit_card
138 SET iban = 'TR323456312213576817699999'
139 WHERE id = 'CcU-2938';
140
141 -- Registro después del cambio
142 • SELECT *
143 FROM credit_card
144 WHERE id='CcU-2938';
145
```



id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30

```
5002 16:17:37 INSERT INTO credit_card (id,iban,pan,pin, cvv, expiring_date) VALUES ('CcU-2938','XX91567... 1 row(s) affected
5003 16:17:37 SELECT FROM Credit_card 5000 row(s) returned
5004 16:17:50 ALTER TABLE transaction ADD constraint FK2 foreign key (credit_card_id) REFERENCES cre... 100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
```



id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30

```
5007 16:19:21 SELECT FROM credit_card 5000 row(s) returned
5008 16:25:53 SELECT FROM transactions credit_card 5000 row(s) returned
5009 16:29:30 SELECT FROM credit_card WHERE id='CcU-2938' 1 row(s) returned
```

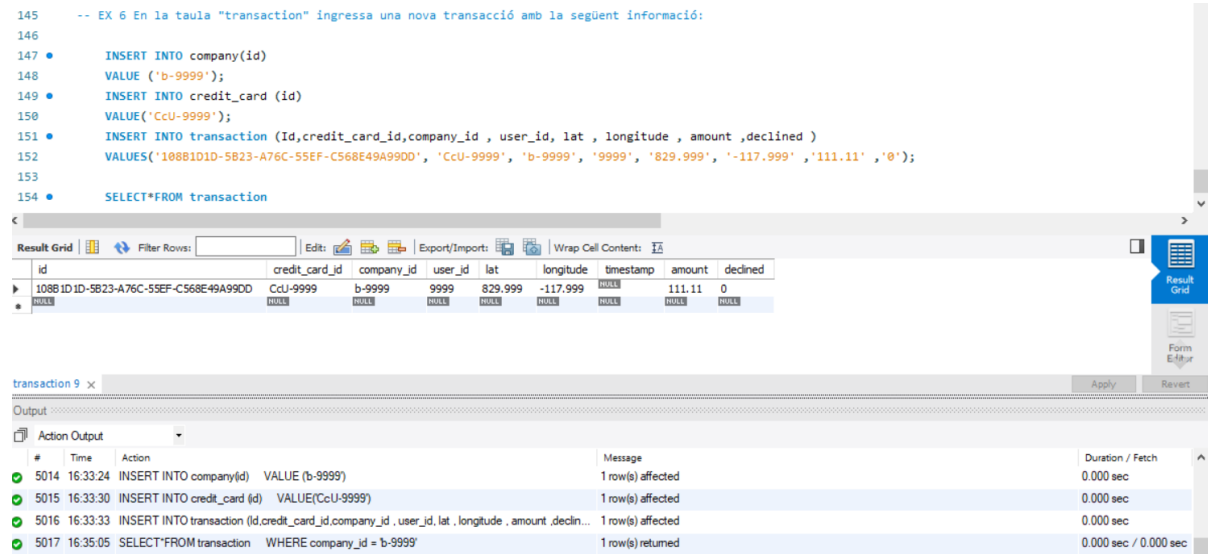
Utilizo UPDATE para aplicar la corrección solicitada por Recursos Humanos.

Establezco el nuevo valor 'TR323456312213576817699999' en la columna iban.

Uso la cláusula WHERE id = 'CcU-2938' para asegurar que el cambio afecta únicamente a este registro y no altera ninguna otra tarjeta del sistema.

EJERCICIO 6

```
145 -- EX 6 En la taula "transaction" ingressa una nova transacció amb la següent informació:
146
147 • INSERT INTO company(id)
148   VALUE ('b-9999');
149 • INSERT INTO credit_card (id)
150   VALUE('CcU-9999');
151 • INSERT INTO transaction (Id,credit_card_id,company_id , user_id, lat , longitude , amount ,declined )
152   VALUES('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', '111.11', '0');
153
154 • SELECT*FROM transaction
```



id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
10881D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	111.11	0	

#	Time	Action	Message	Duration / Fetch
5014	16:33:24	INSERT INTO company(id) VALUE ('b-9999')	1 row(s) affected	0.000 sec
5015	16:33:30	INSERT INTO credit_card (id) VALUE('CcU-9999')	1 row(s) affected	0.000 sec
5016	16:33:33	INSERT INTO transaction (Id,credit_card_id,company_id , user_id, lat , longitude , amount ,declined) VALUES('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', '111.11', '0')	1 row(s) affected	0.000 sec
5017	16:35:05	SELECT*FROM transaction WHERE company_id = 'b-9999'	1 row(s) returned	0.000 sec / 0.000 sec

Al diseñar un modelo en estrella con claves foráneas, no puedo insertar la transacción directamente si las entidades relacionadas (company,credit_card) no existen primero.Sigo el siguiente orden:

Insertar datos en tablas hijas.

Inserto primero el registro en la tabla company con el ID 'B-9999'.

inserto el registro en la tabla credit_card con el ID 'CcU-9999'.

Finalmente, inserto la línea solicitada a la tabla hija, transaction.

EJERCICIO 7

The screenshot displays a database management interface with a SQL script editor and a results pane. The script, labeled 'credit_card 10', contains the following SQL commands:

```
-- Ejercicio 7 Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordra mostrar el canvi realitza
-- Tabla antes del cambio.
SELECT*
FROM credit_card;
-- ejecución
ALTER TABLE credit_card
DROP COLUMN pan;
-- Tras el cambio
SELECT*
FROM credit_card;
```

The results pane shows the state of the 'credit_card' table before and after the execution of the script. The table has columns: id, iban, pan, pin, cvv, and expiring_date. The 'pan' column is present in the initial state and is removed after the execution of the 'ALTER TABLE' statement.

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	2025-09-27
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	2028-12-28
CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	2026-11-26
CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	2027-07-27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	2026-04-25
CcS-4862	XX4774771467463645400768	6716617000907830	4047	174	2026-11-27

The 'Output' pane shows the execution results of the script, including the number of rows returned and the duration of each query.

#	Time	Action	Message	Duration / Fetch
5017	16:35:05	SELECT*FROM transaction WHERE company_id = b-9999'	1 row(s) returned	0.000 sec / 0.000 sec
5018	16:39:23	SELECT* FROM credit_card	5001 row(s) returned	0.000 sec / 0.015 sec
5019	16:39:56	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
5020	16:40:04	SELECT* FROM credit_card	5001 row(s) returned	0.000 sec / 0.000 sec

Uso ALTER TABLE para eliminar permanentemente la columna PAN de la tabla credit_card.

EJERCICIO 8

```
168 -- Ejercicio 8 Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui.
169
170 • CREATE DATABASE IF NOT EXISTS bancos;
171 • USE bancos;
172
173 • DROP TABLE if exists users;
174 • CREATE TABLE users(
175   id INT PRIMARY KEY,
176   name VARCHAR (50),
177   surname VARCHAR (50),
178   phone VARCHAR (20),
179   email VARCHAR(50),
180   birth_date VARCHAR(50),
181   country VARCHAR (50),
182   city VARCHAR (50),
183   postal_code VARCHAR (50),
184   address VARCHAR (100),
185   region VARCHAR (20) NULL
186 );
187
188 • LOAD DATA
189   INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__european_users.csv"
190   INTO TABLE users;
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209 ENCLOSED BY ',';
210 IGNORE 1 ROWS
211 (id,name,surname,phone,email,birth_date,country,city,postal_code,address,@region);
212
213 • UPDATE users
214   SET region = CASE
215     WHEN country IN ('United States','Canada') THEN 'America'
216     ELSE 'Europe'
217   END;
218
```

Output

#	Time	Action	Message	Duration / Fetch
5022	16:44:16	CREATE DATABASE IF NOT EXISTS bancos	1 row(s) affected	0.016 sec
5023	16:44:19	USE bancos	0 row(s) affected	0.000 sec
5024	16:44:29	CREATE TABLE users(id INT PRIMARY KEY, name VARCHAR (50), surname VARCHAR (50), phone VARCHAR (20), email VARCHAR (50), birth_date VARCHAR (50), country VARCHAR (50), city VARCHAR (50), postal_code VARCHAR (50), address VARCHAR (100), region VARCHAR (20) NULL)	0 row(s) affected	0.016 sec
5025	16:44:59	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__europea..."	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.171 sec
5026	16:45:02	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__america..."	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.032 sec

209 ENCLOSED BY ',';

210 IGNORE 1 ROWS

211 (id,name,surname,phone,email,birth_date,country,city,postal_code,address,@region);

212

213 • UPDATE users

214 SET region = CASE

215 WHEN country IN ('United States','Canada') THEN 'America'

216 ELSE 'Europe'

217 END;

218

Result Grid

id	name	surname	phone	email	birth_date	country	city	postal_code	address	region
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	America
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	United States	Philadelphia	19101	903 Sit Ave	America
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	United States	Houston	77001	736-2063 Tellus St.	America
4	Howard	Stafford	1-411-740-3269	ornare.egestas@icloud.edu	Feb 18, 1989	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	America
5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	Sep 26, 1998	United States	Philadelphia	19101	341-2821 Ultrices Av.	America
6	Joel	Tyson	(718) 288-8020	gravida.nunc.sed@yahoo.ca	Oct 15, 1989	United States	San Jose	95101	888-2799 Amet Street	America
7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	Dec 4, 1981	United States	Chicago	60601	8627 Malesuada Rd.	America
8	Nissim	Franks	(692) 157-3469	egestas.aliquam.fringilla@google.ca	Aug 1, 1993	United States	New York	10001	Ap #251-7144 Integer St.	America
9	Mannix	Mcclain	(590) 883-2184	aliquam.nisi@outlook.com	Jan 24, 1987	United States	San Antonio	78201	647-3080 Lacus. St.	America
10	Robert	McCarthy	(324) 746-6771	fermentum@protonmail.com	Apr 30, 1984	United States	San Jose	95101	P.O. Box 773	America

USERS 12

Output

#	Time	Action	Message	Duration / Fetch
5024	16:44:29	CREATE TABLE users(id INT PRIMARY KEY, name VARCHAR (50), surname VARCHAR (50), phone VARCHAR (20), email VARCHAR (50), birth_date VARCHAR (50), country VARCHAR (50), city VARCHAR (50), postal_code VARCHAR (50), address VARCHAR (100), region VARCHAR (20) NULL)	0 row(s) affected	0.016 sec
5025	16:44:59	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__europea..."	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.171 sec
5026	16:45:02	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__america..."	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.032 sec
5027	16:59:41	UPDATE users SET region = CASE WHEN country IN ('United States','Canada') THEN 'America' ELSE 'Europe' END;	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.125 sec
5028	17:00:09	select * FROM USERS	5000 row(s) returned	0.000 sec / 0.016 sec

The screenshot displays a SQL IDE interface with a script editor at the top and an output pane at the bottom. The script editor contains SQL code for creating and loading data into two tables: `companies` and `credit_cards`.

Script Editor Content:

```
226 website VARCHAR (100)
227 );
228
229 LOAD DATA
230 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ companies.csv"
231 INTO TABLE companies
232 FIELDS TERMINATED BY ','
233 ENCLOSED BY '"'
234 IGNORE 1 ROWS
235 (company_id,company_name,phone,email,country,website);
```

Output Pane Content:

Companies Table Data:

company_id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.co.uk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
b-2242	Donec Ltd	01 25 51 37 37	at.laculis@hotmail.co.uk	Norway	https://nytimes.com/user/110
b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one
b-2250	Amet Nulla Donec Corporation	07 15 25 14 74	mattis.integer.eu@protonmail.net	Italy	https://netflix.com/sub/cars
b-2254	Nascetur Ridiculus Mus Inc.	06 26 87 61 84	suspendisse.dui@icloud.net	United States	https://ebay.com/sub
b-2258	Vestibulum Lorem PC	02 02 87 33 40	aenean.massa.integer@aol.net	Belgium	https://pinterest.com/sub/cars

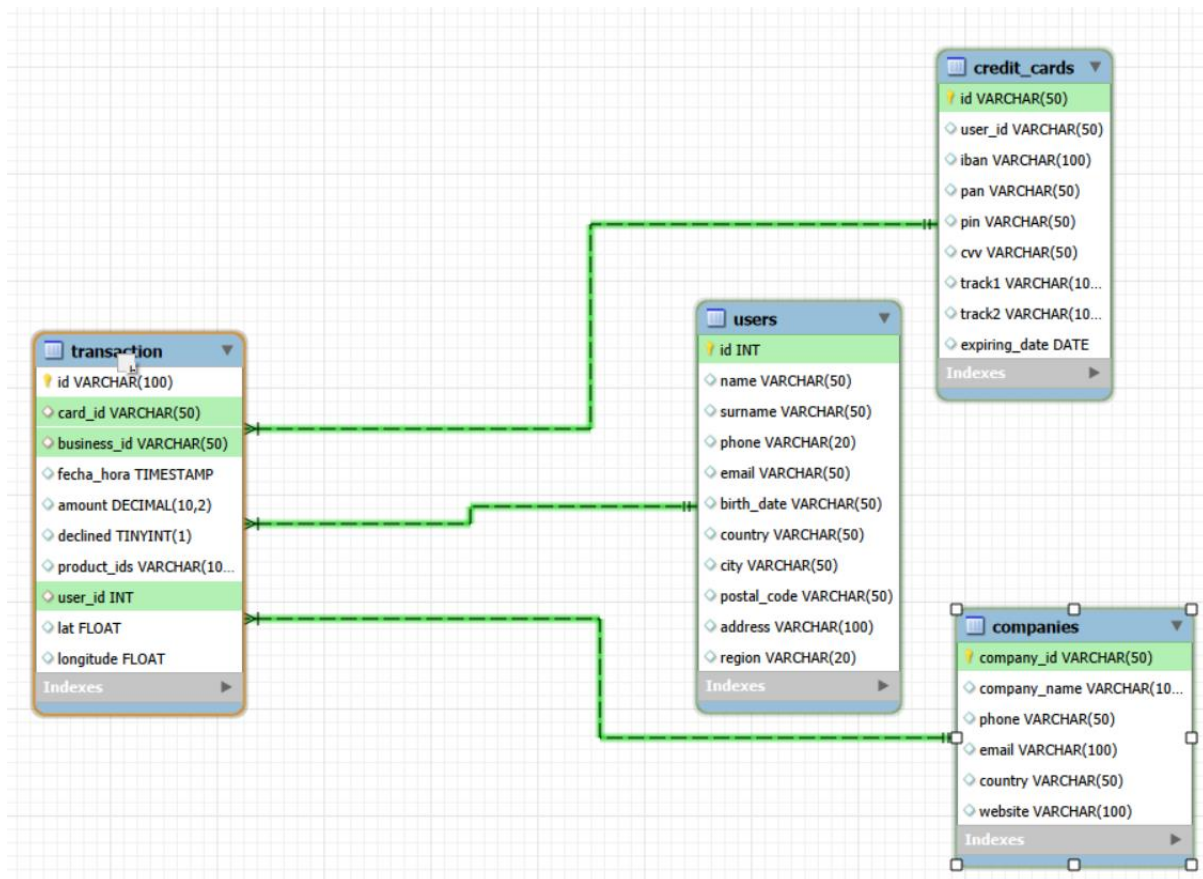
Script Editor Content (Continued):

```
236
237 -- Creación de credit cards
238 CREATE TABLE IF NOT EXISTS credit_cards(
239 id VARCHAR (50) PRIMARY KEY,
240 user_id VARCHAR (50),
241 iban VARCHAR (100),
242 pan VARCHAR (50),
243 pin VARCHAR (50),
244 cvv VARCHAR (50),
245 track1 VARCHAR (100),
246 track2 VARCHAR (100),
247 expiring_date VARCHAR (50),
248 expiring_date_ok DATE NULL);
249
250 LOAD DATA
251 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ credit_cards.csv"
252 INTO TABLE credit_cards
253 FIELDS TERMINATED BY ','
254 ENCLOSED BY '"'
255 IGNORE 1 ROWS
256 (id,user_id,iban,pan,pin,card,track1,track2,expiring_date,@expiring_date_ok);
257
```

Output Pane Content (Continued):

#	Time	Action	Message	Duration / Fetch
5029	17:03:42	CREATE TABLE companies (company_id VARCHAR(50) PRIMARY KEY, company_name ...	0 row(s) affected	0.015 sec
5030	17:03:45	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ compani...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec
5031	17:04:06	SELECT*FROM COMPANIES	100 row(s) returned	0.000 sec / 0.000 sec
5032	17:08:47	CREATE TABLE IF NOT EXISTS credit_cards(id VARCHAR (50) PRIMARY KEY, user_id V...	0 row(s) affected	0.031 sec
5033	17:08:50	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1.Ex.8__ credit_c...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.141 sec

IT Academy



Para este ejercicio, primero creé la base de datos bancos, donde ancle todas las tablas. Identifiqué que, aunque tenía cinco fuentes de datos originales (Transaction, companies, american_users y european_users), las tablas de usuarios americanos y europeos compartían la misma estructura, por lo que decidí unificarlas en una sola tabla de dimensión llamada users. Al analizar el modelo, determiné que transaction era mi tabla de hechos, por lo que seguí el orden lógico de diseño: creé primero las tablas de dimensión (users, companies y credit_cards) y, finalmente, la tabla de hechos, definiendo en ese paso las claves foráneas necesarias para establecer las relaciones.

Para la carga de datos, utilicé LOAD DATA INFILE apuntando a los archivos CSV. Al unificar los usuarios en la tabla users, creé una nueva columna llamada region para preservar el origen de la información y luego la poblé con los valores 'America' o 'Europe' según el país. En el caso de la tabla credit_cards, enfrenté un problema de formato en las fechas de caducidad, así que creé una columna auxiliar temporal expiring_date_ok. Durante la carga, utilicé el prefijo @ en la variable de destino para que el sistema ignorara la escritura directa y evitara errores de tipo. Posteriormente, ejecuté una transformación con STR_TO_DATE para convertir el texto al formato estándar de SQL y, finalmente, eliminé la columna original de texto y renombré la columna limpia, dejando el campo expiring_date listo para consultas eficientes.

EJERCICIO 9

```
294 • SELECT users.name AS Nombre_completo, id
295 FROM users
296 WHERE id in (
297     SELECT user_id
298     FROM transaction
299     GROUP BY user_id
300     HAVING COUNT(transaction.amount)>80);
301
```

Nombre_completo	id
Molly	185
Dxxwgl	289
Bnyr	318
Sfzzoh	454

users 17 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:31:38	SELECT users.name AS Nombre_completo, id FROM users WHERE id in (SELECT user_id FROM transaction GROUP BY user_id HAVING COUNT(transaction.amount)>80);	4 row(s) returned	0.218 sec / 0.000 sec

Realiza una subconsulta que muestre todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

Para este ejercicio, primero ejecuté una subconsulta sobre la tabla `transaction`, donde agrupé los registros por `user_id` y conté el número de operaciones de cada usuario, filtrando únicamente a aquellos que superaban las 80 transacciones utilizando `HAVING`. Con esa lista de identificadores válidos, volví a la tabla de dimensión `users` para seleccionar y mostrar el nombre completo y el ID de los usuarios que coincidían con mi criterio, logrando así cruzar información de dos tablas diferentes para identificar a los clientes más activos sin necesidad de listar cada transacción individualmente.

EJERCICIO 10

Muestra el promedio de importe por IBAN de las tarjetas de crédito en la compañía Donec Ltd, utiliza al menos 2 tablas.

The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
SELECT credit_cards.iban, ROUND(AVG (amount),2)AS media_amount
FROM credit_cards
JOIN transaction ON credit_cards.id = transaction.card_id
JOIN companies ON companies.company_id = transaction.business_id
WHERE companies.company_name = 'Donec Ltd'
GROUP BY credit_cards.iban;
```

The results are displayed in a table with two columns: **iban** and **media_amount**. The data is as follows:

iban	media_amount
XX911406401125586307586805	356.25
SK9446370242474562577506	142.96
XX776752917845952975555640	257.37
XX413827362289719304908990	139.59
XX347787246070769610780308	240.41
XX688768436543090894854602	188.58
MK28368851538688349	439.39
PI 76749783566857676343404576	541.56

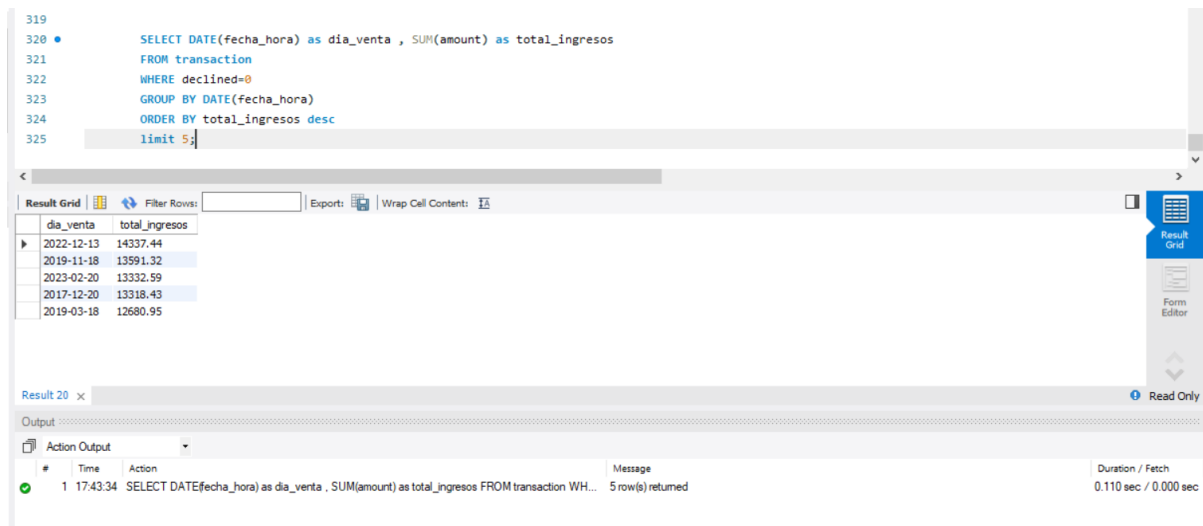
The bottom of the screenshot shows the 'Output' section with two actions:

#	Time	Action	Message	Duration / Fetch
1	17:31:38	SELECT users.name AS Nombre_completo, id FROM users WHERE id in (SELECT user_id FROM ...	4 row(s) returned	0.218 sec / 0.000 sec
2	17:32:10	SELECT credit_cards.iban, ROUND(AVG (amount),2)AS media_amount FROM credit_cards JOIN tr...	371 row(s) returned	0.015 sec / 0.000 sec

Para este ejercicio, vinculé tres tablas (`credit_cards`, `transaction` y `companies`) para analizar el comportamiento de gasto específico de una empresa. Primero, uní las tablas de tarjetas y transacciones mediante sus claves y, luego, conecté el resultado con la tabla de empresas para poder filtrar por nombre. Utilicé una cláusula `WHERE` para seleccionar únicamente las operaciones realizadas en la compañía 'Donec Ltd'. Finalmente, agrupé los datos por `iban` de la tarjeta y calculé el promedio (`AVG`) del `amount` de esas transacciones, redondeando el resultado a dos decimales para obtener un valor claro y legible del gasto medio por tarjeta en dicha empresa.

NIVEL 2

EJERCICIO 1



The screenshot shows a database query editor interface. At the top, a SQL query is entered in a text area:

```
319
320 • SELECT DATE(fecha_hora) as dia_venta , SUM(amount) as total_ingresos
321 FROM transaction
322 WHERE declined=0
323 GROUP BY DATE(fecha_hora)
324 ORDER BY total_ingresos desc
325 limit 5;
```

Below the query editor, the results are displayed in a table with two columns: `dia_venta` and `total_ingresos`. The table contains five rows of data, sorted by total income in descending order.

dia_venta	total_ingresos
2022-12-13	14337.44
2019-11-18	13591.32
2023-02-20	13332.59
2017-12-20	13318.43
2019-03-18	12680.95

At the bottom of the interface, an 'Output' panel shows the execution details of the query:

#	Time	Action	Message	Duration / Fetch
1	17:43:34	SELECT DATE(fecha_hora) as dia_venta , SUM(amount) as total_ingresos FROM transaction WH...	5 row(s) returned	0.110 sec / 0.000 sec

Para este ejercicio, seleccione las transacciones de la tabla `transaction` para identificar los días con mayor generación de ingresos por ventas. Primero, filtré únicamente las operaciones no rechazadas utilizando la cláusula `WHERE declined = 0` para asegurar que solo se cuenten las ventas completas. Luego, agrupé los datos extrayendo solo la fecha completa de la columna `fecha_hora` mediante la función `DATE()`. Finalmente, calculé la suma total de los montos (`SUM(amount)`) para cada día, ordené los resultados de mayor a menor con `ORDER BY` y limité la salida a los cinco registros superiores con `LIMIT 5` para mostrar los días más rentables.

EJERCICIO 2

```
330
331 • SELECT companies.company_name, companies.phone,DATE (transaction.fecha_hora)AS Fecha,transaction.amount
332 FROM companies
333 JOIN transaction
334 On transaction.business_id=companies.company_id
335 WHERE amount BETWEEN 350 AND 400
336 AND DATE(fecha_hora) in ('2015-04-29', '2018-07-20', '2024-03-13')
337 ORDER BY amount DESC;
```

company_name	phone	Fecha	amount
Aliquam PC	01 45 73 52 16	2024-03-13	399.84
Auctor Mauris Vel LLP	08 09 28 74 14	2018-07-20	399.51
At Pede Corp.	06 14 48 33 15	2015-04-29	390.69
Aliquam PC	01 45 73 52 16	2024-03-13	388.29
Ord Adipiscing Limited	03 18 00 77 81	2018-07-20	373.71
Fringilla LLC	08 29 15 93 57	2015-04-29	367.62
Pede Cum Ltd	07 62 26 48 38	2018-07-20	356.87
Auctor Mauris Vel LLP	08 09 28 74 14	2024-03-13	353.75

Result 22 x

Output

#	Time	Action	Message	Duration / Fetch
1	17:45:25	SELECT companies.company_name, companies.phone,DATE (transaction.fecha_hora)AS Fecha,transaction.amount	8 row(s) returned	0.047 sec / 0.000 sec

primero, uní ambas tablas utilizando la clave foránea business_id que conecta con company_id. Luego, filtré los resultados con una cláusula WHERE que exige dos condiciones simultáneas: que el amount esté entre 350 y 400 euros y que la fecha de la transacción coincida exactamente con el 29 de abril de 2015, el 20 de julio de 2018 o el 13 de marzo de 2024. Finalmente, seleccioné el nombre de la empresa, el teléfono, la fecha y el monto, ordenando la lista de mayor a menor cantidad para priorizar las operaciones más grandes dentro de esos criterios.

EJERCICIO 3

```
342
343 • SELECT business_id, COUNT(id) as Cantidad_trasacciones,
344 CASE
345 WHEN COUNT(id) >= 400 THEN 'Mayor 400'
346 ELSE 'Inferior 400'
347 END AS categoria_trasacciones
348 FROM transaction
349 GROUP BY business_id;
```

business_id	Cantidad_trasacciones	categoria_trasacciones
b-2222	2401	Mayor 400
b-2226	410	Mayor 400
b-2230	447	Mayor 400
b-2234	1514	Mayor 400
b-2238	472	Mayor 400
b-2242	449	Mayor 400
b-2246	1541	Mayor 400
b-2250	1511	Mayor 400
b-2254	424	Mayor 400

Result 24 x

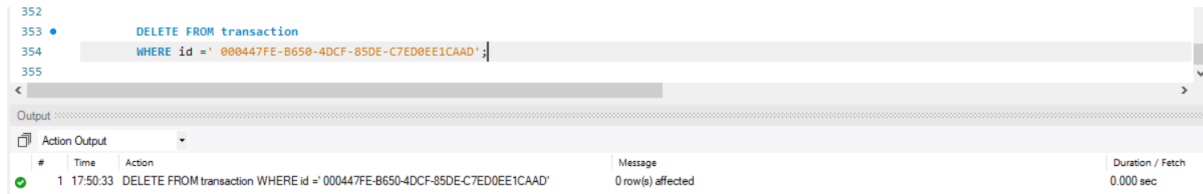
Output

#	Time	Action	Message	Duration / Fetch
1	17:49:02	SELECT business_id, COUNT(id) as Cantidad_trasacciones, CASE WHEN COUNT(id) >= 400 TH...	100 row(s) returned	0.031 sec / 0.000 sec

Para este ejercicio, analicé la tabla transaction para cuantificar la carga operativa de cada empresa y clasificarla según su volumen de actividad. Primero, agrupé los registros por business_id para agrupar todas las operaciones de cada entidad. Luego, utilicé la función COUNT(id) para calcular el número total de transacciones de cada una. Para cumplir con el requisito de Recursos Humanos, implementé una estructura CASE que evalúa ese conteo: si la cantidad es mayor o igual a 400, asigna la categoría

'Mayor 400'; en caso contrario, la etiqueta como 'Inferior 400'. El resultado final muestra el identificador de la empresa, el total de transacciones y su categoría correspondiente para facilitar la asignación de recursos.

EJERCICIO 4



Para resolver este ejercici, utilicé la sentencia DELETE FROM tranasaction para borrar información de esa tabla. Luego, apliqué una cláusula WHERE precisa que filtra únicamente por el id igual al valor '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'.

EJERCIO 5

EJERCICIO 5

1 • `SELECT * FROM bancos.vistamarketing;`

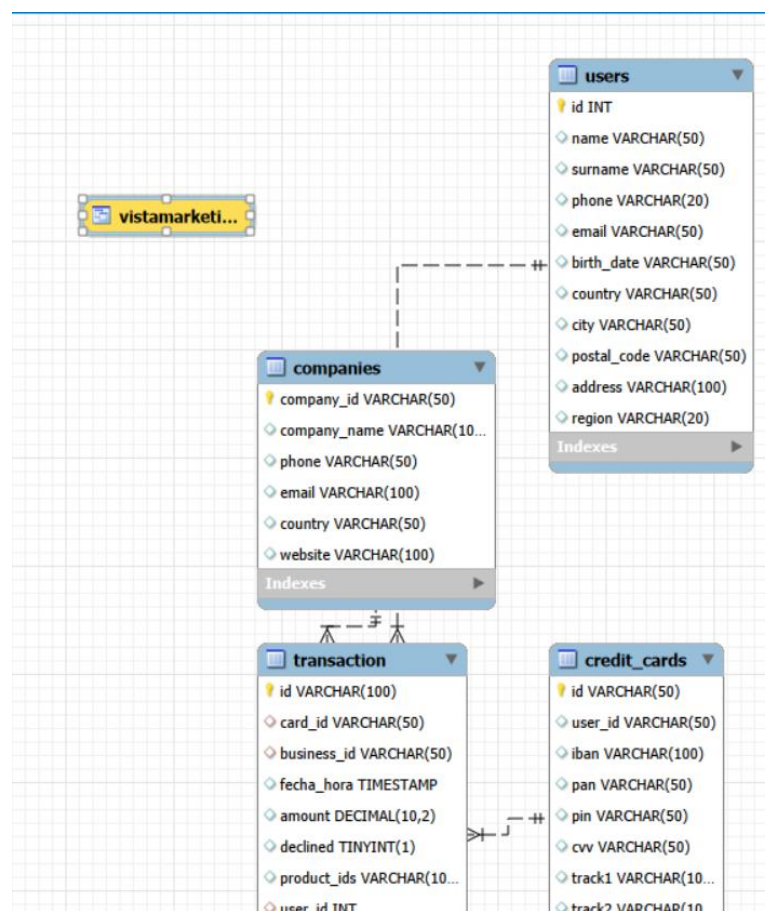
Result Grid

company_name	phone	country	media_compra
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.91
Pretium Neque Corp.	07 77 48 55 28	Australia	275.58
Urna Convalis Associates	06 01 24 77 04	United States	273.57
At Associates	09 56 61 10 65	New Zealand	272.74
Metus Vitae Associates	08 25 44 40 66	Australia	270.05
Aliquet Diam Limited	02 76 61 47 46	United States	269.29
Nec Luctus LLC	02 14 71 75 73	Norway	268.60
Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.56
Cras Consulting	07 50 10 85 63	Belgium	267.38

amarketing 1 x

Output

#	Time	Action	Message	Duration / Fetch
1	17:52:08	CREATE VIEW VistaMarketing AS SELECT company_name ,phone , country ,ROUND(AVG(transa...	0 row(s) affected	0.015 sec
2	17:53:49	SELECT * FROM bancos.vistamarketing	100 row(s) returned	0.313 sec / 0.000 sec



Para este ejercicio, creé la vista 'VistaMarketing' que contiene información de las tablas **companies** y **transaction** para facilitar el análisis de la estrategia de ventas. Primero, definí la estructura de la vista seleccionando el nombre de la compañía, el teléfono y el país, y calculando el promedio de los montos de compra ($\text{AVG}(\text{amount})$) redondeado a dos decimales. Luego, uní las tablas mediante las claves **business_id** y **company_id**, y filtré únicamente las operaciones no rechazadas ($\text{WHERE declined} = 0$) para asegurar

que el promedio refleje ventas reales. Finalmente, agrupé los datos por el identificador de la empresa y ordené el resultado de la vista de mayor a menor media de compra .